

Q1 :- Data Understanding Identify all data quality issues present in the dataset that can cause problems during data loading.

Answer :-

1. Duplicate Records

- **Location:** The rows for **Order_ID O101** (first and fourth rows) are identical.
- **Problem:** This leads to "double-counting" in financial reports, inflating total sales and order counts.

2. Missing Values (Nulls)

- **Location:** **Order_ID O102** has a **Null** value in the **Sales_Amount** column.
- **Problem:** Mathematical operations (like calculating the Average or Sum) may fail or return incorrect results depending on how the loading tool handles nulls.

3. Inconsistent Data Types

- **Location:** **Order_ID O104** has "Three Thousand" written in text, while other rows use numeric values (e.g., 4500).
- **Problem:** This is a major issue for data loading. A database expects a **Numeric** type for sales; a string like "Three Thousand" will cause a "Type Mismatch" error and prevent the data from loading.

4. Inconsistent Date Formats

- **Location:**
 - Most dates use **DD-MM-YYYY** (e.g., 12-01-2024).
 - **Order_ID O103** uses **YYYY/MM/DD** (2024/01/18).
- **Problem:** Most systems require a uniform date format. During the ETL (Extract, Transform, Load) process, the system might misinterpret the day as the month or fail to recognize the entry as a valid date entirely.

Order_ID	Column	Issue Type	Description
O101	All	Duplication	Exact duplicate of the first row.

O102	Sales_Amount	Completeness	Contains a Null value.
O103	Order_Date	Consistency	Different date format (YYYY/MM/DD).
O104	Sales_Amount	Validity	Text value in a numeric column.

Q2. Primary Key Validation

Assume Order_ID is the Primary Key.

a) Is the dataset violating the Primary Key rule?

b) Which record(s) cause this violation?

Answer :- (a). Yes, this dataset is **violating the Primary Key rule**.

In a relational database, a Primary Key must satisfy two main conditions:

1. **Uniqueness:** Every value in the Primary Key column must be unique. No two rows can have the same identifier.
2. **Non-Nullability:** The column cannot contain **Null** values (though that is not the issue here).

Why it violates the rule:

Looking at your dataset, the value **0101** appears twice in the **Order_ID** column:

- **Row 1:** Order_ID **0101**, Customer_ID **C001**, Sales_Amount **4500**, Date **12-01-2024**.
- **Row 4:** Order_ID **0101**, Customer_ID **C001**, Sales_Amount **4500**, Date **12-01-2024**.

Because **0101** is repeated, the **Order_ID** column contains **duplicate values**, which is a direct violation of the Primary Key constraint.

(b). Based on the dataset provided, if **Order_ID** is defined as the **Primary Key**, it must follow two fundamental rules:

1. **Uniqueness:** No two rows can have the same value.
2. **Not Null:** The value cannot be empty (though all cells in that column are filled here).

Records Causing the Violation

The violation is caused by **duplicate entries** for the same Order_ID.

Order_ID	Customer_ID	Sales_Amount	Order_Date
O101	C001	4500	12-01-2024
O101	C001	4500	12-01-2024

Q3 :- Missing Value Analysis.

Which column(s) contain missing values?

- List the affected records.
- Explain why loading these records without handling missing values is risky.

Answer :- (a). Missing Value Analysis

The column that contains a missing value is **Sales_Amount**.

a) Affected Records

In data analysis, a "missing value" is typically represented by a **Null** entry, meaning the data was never recorded or is unknown.

- Order_ID O102:** The **Sales_Amount** for **Customer_ID C002** is explicitly listed as **Null**.

(b). 1. Column(s) Containing Missing Values

In this dataset, the **Sales_Amount** column contains a missing value.

- Row 2 (Order O102):** The **Sales_Amount** is explicitly listed as **"Null"**.

While "Null" is the only technically "missing" value, it's worth noting that Row 5 (Order O104) has `Sales_Amount` written as **"Three Thousand"**. In many automated data loading systems, this text would be treated as invalid or "missing" because it cannot be processed as a number.

2. Risks of Loading Records Without Handling Missing Values

Loading data with missing or inconsistent values into a database or analysis tool can lead to several significant issues:

- **Calculation Errors:** If you try to calculate the total revenue or average sale price, the "Null" value will either cause the entire calculation to fail (error) or lead to an underestimation of the true total.
- **Data Type Mismatches:** Most systems expect the `Sales_Amount` column to be a numeric type (like Integer or Decimal). A "Null" or text like "Three Thousand" can cause the data loading (ETL) process to crash because the data doesn't match the required format.
- **Inaccurate Business Insights:** Missing data leads to "dirty data." Decisions based on incomplete datasets—such as identifying your best customer or forecasting monthly sales—will be skewed and unreliable.
- **Issues with Joins and Relationships:** If a missing value exists in a primary key or foreign key column (though not the case here), it becomes impossible to link that record to other relevant data, such as customer profiles or shipping details.

Q4 :- Data Type Validation

Identify records where `Sales_Amount` violates expected data type rules.

a) Which record(s) will fail numeric validation?

b) What would happen if this dataset is loaded into a SQL table with `Sales_Amount` as DECIMAL?

Answer :- a) Records Failing Numeric Validation

A numeric validation check expects digits (and potentially a decimal point). In this dataset, **two records** will fail:

1. **Order_ID O102:** The value is `Null`. While some systems handle nulls separately, in a strict numeric validation rule (e.g., "Must be a number"), a null entry is often flagged as missing or invalid data.
2. **Order_ID O104:** The value is `"Three Thousand"`. This is a string/text representation of a number, not a numeric data type. It will fail any automated numeric validation.

b) Impact of Loading into a SQL DECIMAL Table

If you attempt to load this raw dataset into a SQL table where **Sales_Amount** is defined as a **DECIMAL** type, the outcome depends on the specific SQL engine and the loading method (e.g., **INSERT** vs. **BULK LOAD**), but generally, the following would occur:

Record	Value	SQL Outcome
O101	4500	Success. Converted to 4500.00 (depending on precision).
O102	Null	Conditional. If the column allows NULLs, it succeeds. If defined as NOT NULL , the load fails.
O103	3200	Success.
O104	Three Thousand	Failure. This will trigger a "Data type mismatch" or "Invalid numeric format" error. In most strict transactions, this would cause the entire batch upload to fail.
O105	5100	Success.

Q5 :- Date Format Consistency

The Order_Date column has multiple formats.

- a) List all date formats present in the dataset
- b) Why is this a problem during data loading?

Answer :- a) Date formats present in the dataset

There are two distinct date formats used in the **Order_Date** column:

1. **DD-MM-YYYY**: Used in most rows (e.g., 12-01-2024, 15-01-2024, 20-01-2024).
2. **YYYY/MM/DD**: Used in the third row for Order_ID 0103 (2024/01/18).

b) Why is this a problem during data loading?

Inconsistent date formats create several critical issues during the ETL (Extract, Transform, Load) process:

- **Parsing Failures**: Most database systems and data tools (like SQL, Power BI, or Python/Pandas) expect a uniform format for a column. If the loader is set to expect DD-MM-YYYY and encounters YYYY/MM/DD, it may result in an error or load the value as a "Null" or "Error" type.
- **Data Integrity Risks**: If the system doesn't crash, it might misinterpret the data. For example, a system might confuse days and months (e.g., reading 01-05-2024 as January 5th instead of May 1st), leading to incorrect business insights.
- **Sorting and Filtering Issues**: Dates stored as strings in different formats will not sort chronologically. 2024/01/18 would be sorted based on alphabetical/numerical string order rather than its actual place in time relative to 12-01-2024.
- **Failed Calculations**: You cannot perform date math (like calculating the number of days between two orders) if the system cannot recognize the entries as valid, standardized "Date" objects.

Q6 :- Load Readiness Decision

Based on the dataset condition:

- a) Should this dataset be loaded directly into the database? (Yes/No)
- b) Justify your answer with at least three reasons

Answer :- a) Should this dataset be loaded directly into the database?

No.

b) Justification

The dataset contains several data quality issues that violate standard database integrity and consistency rules. It requires significant **data cleaning** and **transformation** before it is ready for loading.

1. **Duplicate Records:** The first and fourth rows are identical (Order_ID: O101, Customer_ID: C001, Sales_Amount: 4500). In a relational database, the **Order_ID** should typically be a **Primary Key**, which must be unique. Loading this directly would cause a primary key violation or result in redundant data.
2. **Data Type Inconsistency (Mixed Formats):** The **Sales_Amount** column contains mixed data types. Most entries are numeric (4500, 3200), but one entry is written as text ("Three Thousand"). Most database schemas require a specific data type (like **INT** or **DECIMAL**) for such columns; "Three Thousand" would cause a **data type mismatch error**.
3. **Inconsistent Date Formats:** The **Order_Date** column lacks a uniform format. Most entries follow the **DD-MM-YYYY** format (e.g., 12-01-2024), but the third row uses a **YYYY/MM/DD** format (2024/01/18). This inconsistency can break date-based queries, sorting, and reporting.
4. **Missing Values (Nulls):** The second row (Order_ID: O102) has a **Null** value for **Sales_Amount**. Depending on the business logic or database constraints (such as a **NOT NULL** constraint), this could cause a load failure or lead to inaccurate financial calculations (like a sum or average) if not handled properly.

Q7 :- Pre-Load Validation Checklist List the exact pre-load validation checks you would perform on this dataset before loading.

Answer :- 1. Duplicate Record Check

The dataset contains **redundant information** that could skew sales totals or customer counts.

- **Action:** Identify and remove exact row duplicates.
- **Specific Issue:** The row for **Order_ID O101** appears twice (Row 1 and Row 4) with identical data.

2. Missing Value (Null) Validation

Databases often require numerical fields for calculations; "Null" values can break summation scripts.

- **Action:** Decide on a handling strategy (e.g., replace with 0, impute based on average, or flag for manual review).
- **Specific Issue:** **Order_ID O102** has a **Null** value in the **Sales_Amount** column.

3. Data Type Consistency

A column should contain only one data type (e.g., Integer, String, or Date) to be processed correctly by analytical tools.

- **Action:** Convert text-based numbers into integers or floats.
- **Specific Issue:** `Order_ID O104` has `Sales_Amount` written as "Three Thousand" instead of `3000`. This will cause a "Type Mismatch" error in most systems.

4. Date Format Standardization

Inconsistent date formats prevent chronological sorting and time-series analysis.

- **Action:** Reformat all entries to a standard ISO format (e.g., `YYYY-MM-DD`).
- **Specific Issue:** Most dates follow the `DD-MM-YYYY` format, but `O103` follows `YYYY/MM/DD`.

5. Primary Key Integrity

`Order_ID` should ideally be a unique identifier.

- **Action:** Check for "Logical Duplicates" where the ID is the same but the data differs (though in this specific case, it is a literal duplicate).

Q8 :- Cleaning Strategy Describe the step-by-step cleaning actions required to make this dataset load-ready.

Answer :- . Remove Duplicate Records

The first step is to ensure data integrity by removing redundant entries.

- **Observation:** The first row (`Order_ID O101`) and the fourth row are identical.
- **Action:** Delete the duplicate record so that each `Order_ID` is unique.

2. Standardize Date Formats

Inconsistent date formats can crash data pipelines or cause errors in time-series analysis.

- **Observation:** Most dates follow the `DD-MM-YYYY` format, but row 3 (`Order_ID O103`) uses `YYYY/MM/DD`.
- **Action:** Convert all entries in the `Order_Date` column to a standard ISO format (e.g., `YYYY-MM-DD`).

3. Handle Missing Values (Nulls)

Null values can prevent mathematical operations and aggregations.

- **Observation:** Row 2 (`Order_ID O102`) has a `Null` in the `Sales_Amount` column.

- **Action:** Depending on the business requirements, you should either:
 - **Impute:** Replace `Null` with the mean/median of the column.
 - **Flag:** Replace with `0` if that is a valid business state.
 - **Remove:** Delete the row if the sales amount is critical and cannot be estimated.

4. Correct Data Types & Inconsistent Entries

Columns should contain only one type of data (numeric, string, etc.).

- **Observation:** Row 5 (Order_ID O104) has "Three Thousand" written as text instead of a number.
- **Action:** Convert the string "Three Thousand" to the integer `3000`. Ensure the entire `Sales_Amount` column is set to a numeric data type (Integer or Float).

Q9 :- Loading Strategy Selection

Assume this dataset represents daily sales data.

- a) Should a Full Load or Incremental Load be used?
- b) Justify your choice.

Answer :- Recommended Strategy: Incremental Load

a) Choice

An **Incremental Load** (also known as a Delta Load) should be used for this daily sales data.

b) Justification

The choice of an incremental load is based on the nature of the data and standard Data Warehousing best practices:

- **Data Volume & Frequency:** Since this is "daily sales data," the volume will grow significantly over time. Re-loading the entire history every day (Full Load) would eventually become slow, resource-intensive, and costly.
- **Efficiency:** Incremental loading only captures new records (like a new `Order_ID`) or updates that occurred since the last load. This reduces the processing window and network traffic.
- **Unique Identifiers:** The dataset contains clear keys, such as `Order_ID` and `Order_Date`, which allow the system to identify exactly which records are new or have changed.

- **Historical Tracking:** Sales data is typically transactional. Once an order is placed, it usually stays in the system. An incremental load allows you to append these new transactions to the historical archive without disturbing existing, processed data.

Data Quality Note

Looking at your specific dataset, there are several **data quality issues** (Dirty Data) that would need to be addressed in the **Transformation (ETL)** phase before the load happens:

1. **Duplicates:** Order 0101 appears twice.
2. **Missing Values:** Order 0102 has a Null sales amount.
3. **Data Type Mismatch:** Order 0104 has the amount written as "Three Thousand" (string) instead of a number.
4. **Inconsistent Formatting:** The Order_Date for 0103 uses a different date format (YYYY/MM/DD) compared to the others.

Q10 :- BI Impact Scenario

Assume this dataset was loaded without cleaning and connected to a BI dashboard.

- a) What incorrect results might appear in Total Sales KPI?
- b) Which records specifically would cause misleading insights?
- c) Why would BI tools not detect these issues automatically?

Answer :- a) What incorrect results might appear in Total Sales KPI?

The **Total Sales KPI** would be **understated** (lower than the actual value).

- **Calculation Failures:** Most BI tools perform a SUM() function on numerical columns. Since "Null" and "Three Thousand" are non-numeric, they would be excluded from the mathematical calculation or cause the entire calculation to error out, depending on the tool's strictness.
- **Duplicate Inflation:** Conversely, because Order 0101 appears twice, the dashboard might double-count that specific sale if no unique constraints are applied, potentially inflating the total.

b) Which records specifically would cause misleading insights?

Order_ID	Issue Found	Specific Impact

O101	Duplicate Record	Appears twice (Row 1 and Row 4). This causes double-counting of sales and customer activity.
O102	Missing Value (Null)	The Sales_Amount is empty. The dashboard treats this as \$0\$ sales, which is likely inaccurate.
O103	Format Inconsistency	The Order_Date is YYYY/MM/DD, while others are DD-MM-YYYY. This can break time-series filters (e.g., Year-over-Year growth).
O104	Data Type Mismatch	Sales_Amount is "Three Thousand" (string) instead of 3000 (integer). BI tools cannot sum text, so this revenue "disappears" from the KPI.

c) Why would BI tools not detect these issues automatically?

BI tools are designed for **visualization**, not inherently for **data auditing**. They often follow the "Garbage In, Garbage Out" principle for three main reasons:

1. **Schema Flexibility:** BI tools try to be helpful by guessing data types. If a column has mostly numbers but one "Three Thousand," it might cast the whole column as "Text" to avoid an error, which prevents any math from being done.
2. **Lack of Context:** A tool doesn't know if **O101** appearing twice is a mistake or two separate transactions with the same ID unless a "Unique Constraint" is defined in the underlying database.
3. **Silent Failures:** To prevent a dashboard from "crashing" for the end-user, many tools simply ignore "Nulls" or "Strings" in a numeric column, leading to a successful (but incorrect) visualization.